

Lab Exercise 1- Implementing a Simple API using Java and Testing with Postman

1. Aim

To create a **simple REST API using Java** in **VS Code** and test the API endpoints using **Postman**.

2. Software Requirements

- **Java JDK 17 or later**
- **VS Code**
- **Postman**
- **Maven**
- **Spring Boot Extension Pack** (VS Code)

Recommended tools:

- Visual Studio Code
 - Postman
 - Java Development Kit
 - Spring Boot
-

3. Theory

An **API (Application Programming Interface)** allows different software applications to communicate with each other.

A **REST API** uses HTTP methods such as:

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Remove data

In this experiment, we will build a **simple Student API** with GET and POST operations.

4. Procedure

Step 1: Create a Spring Boot Project

Spring Initializr: **Create a Maven Project**

Choose:

Language: **Java 17**

Spring Boot Version: **Latest Stable**

Group: **com.example**

Artifact: **studentapi**

Dependencies: **Spring Web**

Type: **jar**

Open the generated project in VS Code.

Step 2: Create Student Model

Create a new file:

Student.java

```
package com.example.studentapi;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getCourse() {
```

```
    return course;
}
}
```

Step 3: Create Controller

Create:

```
StudentController.java
```

```
package com.example.studentapi;

import org.springframework.web.bind.annotation.*;
import java.util.*;

@RestController
public class StudentController {

    List<Student> students = new ArrayList<>();

    @GetMapping("/students")
    public List<Student> getStudents() {
        return students;
    }

    @PostMapping("/students")
    public String addStudent(@RequestBody Student student) {
        students.add(student);
        return "Student added successfully";
    }
}
```

```
}  
}
```

Step 4: Run the Application

Open terminal in VS Code and run:

```
mvn spring-boot:run
```

Server starts at:

```
http://localhost:8080
```

5. Testing API using Postman

Open **Postman**.

Test GET Request

Method:

GET

URL:

http://localhost:8080/students

Click **Send**.

Expected Output:

[]

Test POST Request

Method:

POST

URL:

http://localhost:8080/students

Body → Raw → JSON

```
{  
  "id":1,  
  "name":"Rahul",  
  "course":"Computer Science"  
}
```

Click **Send**.

Response:

Student added successfully

Test GET Again

GET http://localhost:8080/students

Response:

```
[
{
  "id":1,
  "name":"Rahul",
  "course":"Computer Science"
}
]
```

6. Expected Result

The **REST API** was successfully created using **Java Spring Boot** in **VS Code** and tested using **Postman**.

7. Conclusion

This experiment demonstrated how to:

- Create a **REST API using Java**
- Run it in **VS Code**
- Test endpoints using **Postman**